

Indtjeningssystem



Velkommen til indtjeningssystemet

Din forretnings samlede indtjening

113992.0 kr

Tilføj produkt

Sammenlign kategorier

Alle produkter

Nuftalem Kibrom Rufael

3.r

Programering B

Mikkel Kirkeby Mosthaf

27.03.26

Slotshaven Gymnasium

Resumé

Dette projekt har til formål at udvikle en webbaseret applikation, der kan hjælpe virksomheder med at analysere og sammenligne indtjening på tværs af produktkategorier.

Applikationen er udviklet med Python og Flask og anvender en SQLite database til lagring af data. Brugeren kan tilføje og se produkter samt sammenligne kategorier ud fra indtjening, antal solgte og gennemsnitspris.

Resultatet er en funktionel webapplikation med et brugervenligt interface og visuelle grafer, der understøtter dataanalyse. Projektet demonstrerer anvendelse af databaser, webudvikling og databehandling.

Indholdsfortegnelse

Resumé	2
Indledning	1
Krav til løsning.....	1
Projektbeskrivelse	2
Fokusområde.....	3
CRUD-operationer	3
Databehandling.....	3
Strukturering af kode	3
Gruppemedlemmernes fokusområder.....	3
Funktionsbeskrivelse - (Skitser og Designproces)	4
Dokumentation	7
Kodegennemgang.....	8
CRUD-operationer.....	8
Databehandling	9
Strukturering af kode.....	10
Brug af AI.....	11
Test.....	12
Konklusion.....	13
Bilag.....	13

Indledning

I dette projekt har vi udviklet en webbaseret applikation, der fungerer som et værktøj til at analysere og sammenligne en virksomheds indtjening. Systemet giver brugeren mulighed for at få overblik over produkter og deres økonomiske betydning på tværs af forskellige kategorier.

Som case har vi taget udgangspunkt i en virksomhed inspireret af Amazon, dog uden anvendelse af live data. Formålet har været at simulere et realistisk system, hvor virksomheder kan arbejde med deres salgsdata.

Problemformulering

Hvordan kan vi udvikle en webbaseret applikation, der ved hjælp af databasehåndtering og databehandling kan hjælpe en virksomhed med at analysere og sammenligne indtjening på tværs af produktkategorier?

Krav til løsning

For at besvare problemformuleringen blev der opstillet en række krav til systemet. Applikationen skulle kunne håndtere og analysere produktdata ved hjælp af databasehåndtering og databehandling. Dette indebærer følgende funktionelle krav:

- | |
|--|
| <i>Brugeren skal kunne tilføje produkter til systemet med oplysninger om navn, kategori, pris og antal solgt</i> |
| <i>Brugeren skal kunne slette produkter fra databasen</i> |
| <i>Systemet skal kunne vise alle produkter struktureret efter kategori</i> |
| <i>Brugeren skal kunne søge efter produkter eller kategorier</i> |
| <i>Systemet skal kunne beregne indtjening baseret på pris og antal</i> |
| <i>Brugeren skal kunne sammenligne to kategorier med hensyn til</i> <ul style="list-style-type: none">- Samlet indtjening- Antal solgte produkter- Gennemsnitspris |

Derudover blev der opstillet følgende ikke-funktionelle krav:

- | |
|---|
| <i>Systemet skal være brugervenligt og overskueligt</i> |
| <i>Data skal præsenteres struktureret i tabeller og grafer</i> |
| <i>Koden skal være opdelt efter trelagsmodellen for at sikre god struktur</i> |

Projektbeskrivelse

Applikationen er udviklet efter trelagsmodellen, hvor systemet opdeles i datalag, logiklag og præsentationslag.

Datalag

Datalaget er implementeret i database.py og består af en relationel SQLite-database. Databasen indeholder følgende information om hvert produkt:

<i>Produktnavn</i>
<i>Kategori</i>
<i>Pris</i>
<i>Antal Solgt</i>

Data gemmes og hentes gennem SQL-forespørgsler. Og indtjening beregnes som:

$$\text{Indtjening} = \text{Pris} \times \text{Antal}$$

Logiklag

Logiklaget er implementeret i app.py ved hjælp af Flask. Dette lag håndterer:

<i>Beregning af samlet indtjening</i>
<i>Summering af indtjening pr. kategori</i>
<i>Sammenligning mellem to kategorier</i>
<i>Validering af brugerinput</i>

Der er også implementeret en klasse (Product), som repræsenterer et produkt og indeholder metoder til beregning af indtjening.

Præsentationslag

Præsentationslaget består af HTML og CSS og udgør brugergrænsefladen. Brugeren har mulighed for at:

<i>Tilføj Produkter</i>
<i>Slette Produkter</i>
<i>Se alle produkter opdelt i kategorier</i>
<i>Søge efter produkter eller kategorier</i>
<i>Sammenligne to kategorier</i>

Data præsenteres i tabeller og visuelle grafer, hvilket gør det nemt at analysere forskelle mellem kategorier.

Fokusområde

I dette projekt har jeg haft særligt fokus på følgende områder:

CRUD-operationer

Jeg har arbejdet med implementering af CRUD-operationer (Create, Read, Update, Delete), som er centrale for håndtering af data i systemet. Fokus har været på at sikre korrekt kommunikation mellem brugergrænsefladen og databasen.

Dette indebærer oprettelse af nye produkter, visning af eksisterende data, samt sletning af produkter fra databasen. Disse operationer er nødvendige for, at brugeren kan interagere med systemet og opdatere virksomhedens data løbende.

Databehandling

Jeg har arbejdet med behandling og bearbejdning af data med henblik på at skabe overblik og sammenligninger. Derudover har jeg arbejdet med at strukturere data, så de kan anvendes i grafer og visuelle præsentationer.

Dette omfatter blandt andet beregning af indtjening, summering af data inden for kategorier samt udregning af gennemsnitsværdier. Formålet har været at omdanne rå data til brugbar information, som kan anvendes til analyse.

Strukturering af kode

Jeg har også haft fokus på at strukturere koden på en overskuelig og vedligeholdelsesvenlig måde. Dette er blandt andet gjort ved at opdele programmet i flere lag efter trelagsmodellen, hvor datalaget og logiklaget er adskilt i forskellige filer. Formålet med denne opdeling er at gøre koden mere modulær og lettere at udvide og fejlrette.

Gruppemedlemmernes fokusområder

Nuftalem Kibrom Rufael
<i>CRUD-operationer (Create, Read, Update, Delete)</i>
<i>Databehandling</i>
<i>Strukturering af kode (funktioner/klasser)</i>

Oliver Heinz Tranberg van Heezewijk
<i>Pivot-lignende sammenligning af kategorier</i>
<i>Database-design (relationel database)</i>
<i>Frontend-udvikling</i>

Funktionsbeskrivelse - (Skitser og Designproces)

I starten af projektet lavede vi nogle skitser af hjemmesiden for at få en ide om, hvordan systemet skulle se ud og fungere. Skitserne gjorde det lettere at planlægge layoutet og de forskellige funktioner, før vi begyndte at kode.

Den første skitse viser en oversigt over produkter i en tabel med navn, kategori, pris og antal solgt. Der er også en samlet indtjening, som giver brugeren et hurtigt overblik over data.

Produkter

Navn	kategori	Pris	Antal solgt
Lanlop	Elektronik	5000 kr.	220
Bog	Bøger	200 kr.	300
Kamera	Elektronik	2500 kr.	50

Samlet indtjening
4.200.000

Søg ... Søg

Figur 1 Oversigt over produkter

Den anden skitse viser siden til at tilføje et produkt. Her kan brugeren indtaste navn, vælge kategori og skrive pris og antal. Skitsen gjorde det nemmere at lave en simpel og overskuelig formular.

tilføj produkt

Produkt navn:

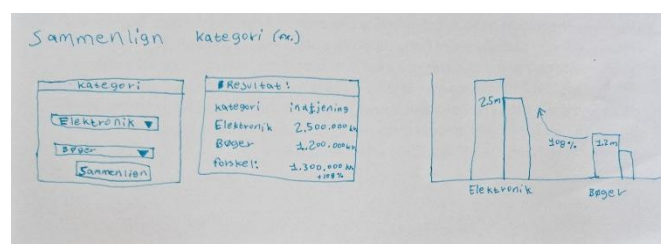
Kategori:

Pris: kr

Antal Solgt:

Figur 2 Tilføjelse af produkt

Den tredje skitse viser sammenligningssiden, hvor man kan vælge to kategorier og sammenligne deres indtjening. Der er også tegnet en graf, som hjælper med at visualisere forskellen mellem kategorierne.



Figur 3 Sammenligningssiden

Programmet er en webbaseret applikation, der giver brugeren mulighed for at håndtere og analysere produktdata gennem en grafisk brugergrænseflade. Applikationen består af flere sider, som hver har deres funktion.

Forside

Forsiden fungerer som navigation og giver adgang til systemets centrale funktioner. Herfra kan brugeren vælge at tilføje produkter, se alle produkter eller sammenligne kategorier.



Figur 4 Forsiden

Tilføj produkt

På denne side kan brugeren indtaste oplysninger om et produkt, herunder navn, kategori, pris og antal. Når formularen sendes, gemmes data i databasen.

Figur 5 Tilføj produkt siden

Alle produkter

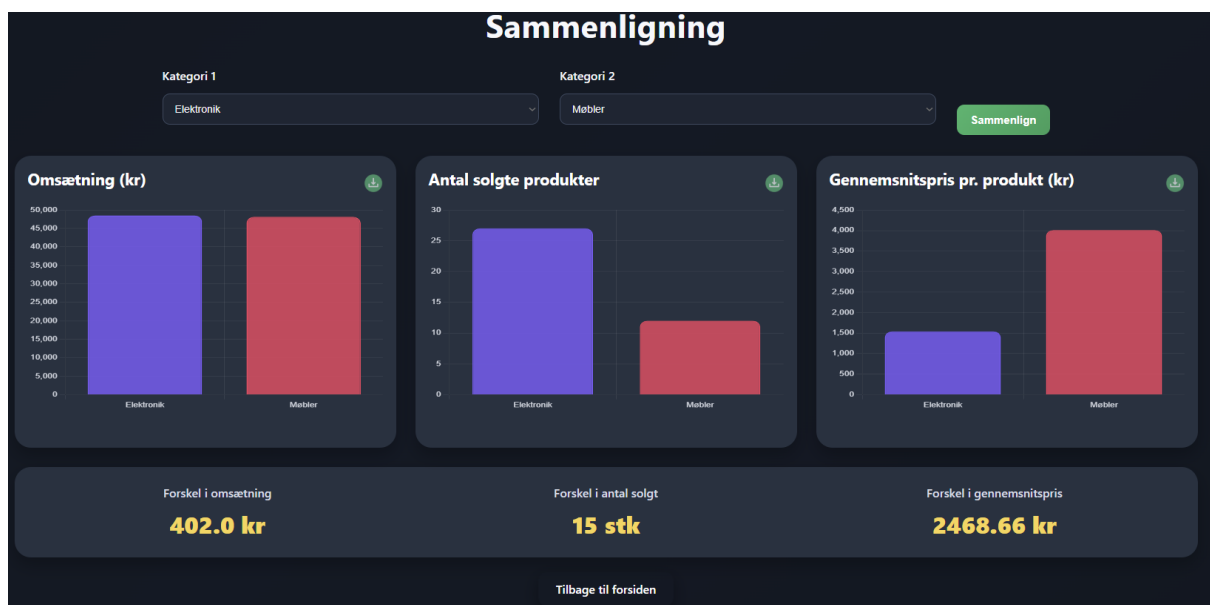
Denne side viser alle produkter opdelt i kategorier (Elektronik, Møbler, Personlig pleje og Tøj). Produkterne vises i tabeller, hvor brugeren kan se relevante oplysninger samt den beregnede indtjening. Brugeren kan også søge efter produkter eller kategorier. Systemet filtrerer data baseret på brugerens input og viser relevante resultater.



Figur 6 Alle produkter siden

Sammenligning

Brugeren kan vælge to kategorier og sammenligne dem. Systemet beregner og viser samlet indtjening, antal solgte produkter og gennemsnitspris. Resultaterne vises både numerisk og visuelt ved hjælp af grafer, hvilket gør det lettere at analysere forskelle mellem kategorier.



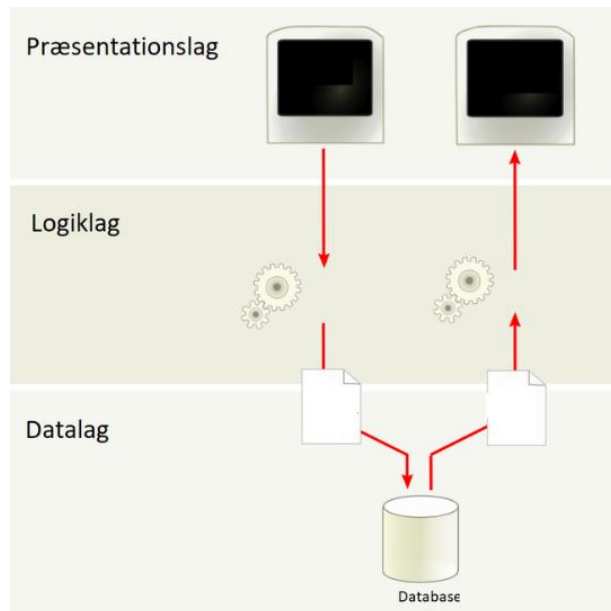
Figur 7 Sammenligning siden

Brugergrænsefladen er designet med fokus på overskuelighed og brugervenlighed. Navigationen er simpel, og data præsenteres struktureret i tabeller og grafer.

Dokumentation

Overordnet opbygning og struktur

Systemet er opbygget efter trelagsmodellen, som opdeler programmet i tre lag.



Præsentationslag (HTML/CSS)

Dette lag håndterer brugergrænsefladen og viser data til brugeren. Brugeren interagerer med systemet gennem formularer og visninger i browseren.

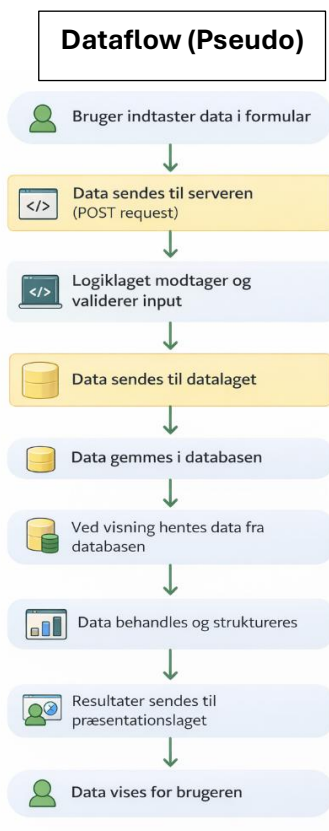
Logiklag (Flask / app.py)

Dette lag håndterer programmets logik, herunder beregninger, routing og behandling af brugerinput. Logiklaget fungerer som bindeled mellem brugergrænsefladen og databasen.

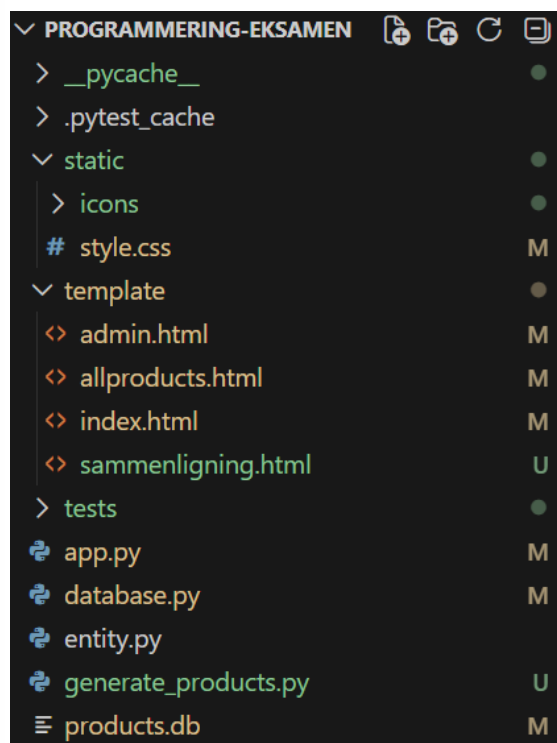
Datalag (database.py / SQLite)

Dette lag håndterer lagring og hentning af data. Alle databaseoperationer udføres her ved hjælp af SQL-forespørgsler.

Diagrammer



Struktur Diagram



Kodegennemgang

I dette afsnit gennemgås centrale dele af programmet med fokus på CRUD-operationer, databehandling og strukturering af kode som var mine fokuspunkter.

CRUD-operationer

Oprettelse af produkt

```
def insert_product(product):  
    conn = get_db()  
    cursor = conn.cursor()  
  
    cursor.execute("""  
        INSERT INTO products (name, category, price, quantity)  
        VALUES (?, ?, ?, ?)  
    """, (product.name, product.category, product.price, product.quantity))  
  
    conn.commit()  
    conn.close()
```

Denne funktion indsætter et nyt produkt i databasen. Data modtages fra brugerens input via en formular i præsentationslaget og sendes til logiklaget, hvor de pakkes ind i et Product-objekt. Herefter videregendes data til datalaget, hvor SQL-forespørgslen udføres.

Brugen af parameteriserede queries (?) sikrer, at systemet er beskyttet mod SQL-injection.

Sletning af produkt

```
def delete_product_db(product_id):  
    conn = get_db()  
    cursor = conn.cursor()  
  
    cursor.execute("DELETE FROM products WHERE id = ?", (product_id,))  
  
    conn.commit()  
    conn.close()
```

Denne funktion fjerner et produkt fra databasen baseret på dets id. Funktionen kaldes fra logiklaget, når brugeren vælger at slette et produkt via brugergrænsefladen.

Disse funktioner viser, hvordan systemet understøtter centrale CRUD-operationer.

Ud over oprettelse og sletning arbejder programmet også med læsning og filtrering af data:

```
def search_products(query):  
    conn = get_db()  
    cursor = conn.cursor()  
  
    cursor.execute("""  
        SELECT * FROM products  
        WHERE name LIKE ? OR category LIKE ?  
    """, (f"%{query}%", f"%{query}%"))
```

Denne funktion gør det muligt at søge i databasen. Her anvendes LIKE til at finde delvise matches, hvilket betyder, at brugeren ikke behøver at skrive hele produktnavnet.

Databehandling

En vigtig del af programmet er behandling af data, hvor rå data fra databasen omdannes til brugbar information.

```
products1 = [Product(*p[1:]) for p in rows1]
```

Her omdannes data fra databasen til objekter ved hjælp af Product-klassen. Dette gør det muligt at arbejde mere struktureret med data og anvende metoder direkte på objekterne.

Beregning af værdier

```
sold1 = sum(p.quantity for p in products1)
```

Denne kode beregner det samlede antal solgte produkter ved at summere quantity for alle produkter i listen.

```
revenue1 = round(sum(p.revenue() for p in products1), 2)
```

Her beregnes den samlede indtjening ved hjælp af metoden revenue(), som returnerer pris gange antal. Funktionen round() anvendes til at sikre en passende præcision.

Ud over de grundlæggende beregninger håndterer programmet også forskelle mellem kategorier:

```
result = {  
    "revenue_difference": abs(revenue1 - revenue2),  
    "sold_difference": abs(sold1 - sold2),  
    "avg_price_difference": abs(avg_price1 - avg_price2)  
}
```

Her anvendes funktionen abs() til at beregne forskellen mellem værdier. Funktionen sikrer, at resultatet altid er positivt, uanset hvilken kategori der har den største værdi.

Programmet tager også højde for situationer, hvor der ikke er nogen data:

```
avg_price1 = round(
    sum(p.price for p in products1) / len(products1), 2
) if products1 else 0
```

Her bruges en betingelse (if products1 else 0) for at undgå en fejl, hvis listen er tom. Uden denne betingelse ville programmet forsøge at dividere med 0, hvilket ville give en runtime-fejl.

Strukturering af kode

Programmet er struktureret efter trelagsmodellen, hvor ansvar er opdelt mellem forskellige filer.

```
def get_products_by_category(category):
    conn = get_db()
    cursor = conn.cursor()

    cursor.execute("""
        SELECT * FROM products
        WHERE category = ?
        ORDER BY name ASC
    """, (category,))

    data = cursor.fetchall()
```

Denne funktion er placeret i database.py og håndterer udelukkende databaseadgang. Logiklaget (app.py) kalder denne funktion uden selv at indeholde SQL-kode. Denne opdeling betyder, at:

- Datalaget håndterer al databasekommunikation
- logiklaget håndterer beregninger og flow
- præsentationslaget håndterer visning

Dette gør systemet mere og lettere at vedligeholde, da ændringer i et lag ikke nødvendigvis påvirker de andre lag.

En vigtig del af struktureringen er, hvordan data bevæger sig gennem systemet:

```
name = request.form["name"]
category = request.form["category"]
price = float(request.form["price"])
quantity = int(request.form["quantity"])
```

Her modtages data fra brugerens input i logiklaget. Data bliver derefter behandlet og sendt videre til datalaget. Dette viser, hvordan programmet følger trelagsmodellen i praksis, hvor input går fra præsentationslaget til logiklaget og videre til databasen.

Programmet er opdelt i funktioner, som hver har et ansvar:

```
def get_products_by_category(category):
```

Ved at opdele koden i mindre funktioner bliver den mere overskuelig og lettere at genbruge. Funktionen kan f.eks. bruges flere steder i programmet uden at gentage kode.

Brug af AI

I dette projekt har vi anvendt kunstig intelligens (AI) som et værktøj til at understøtte udviklingen af vores webapplikation. AI er blevet brugt til at få inspiration, løse tekniske udfordringer samt forbedre design og struktur i vores kode.

Prompt	Formål	Hvordan vi brugte det
Vi gav AI en samlet instruktionsfil om at udvikle en webapplikation i Flask til at starte med	At skabe en samlet struktur og retning for hele projektet	Vi brugte AI's forslag som udgangspunkt for projektets opbygning og videreudviklede selv funktionerne trin for trin
Lav en Flask app med SQLite database, hvor man kan tilføje, vise og slette produkter	At opbygge backend og database	Vi brugte det som grundstruktur og tilpassede koden til vores egen app
Hvordan kan jeg sortere produkter alfabetisk i Flask med SQLite	At gøre data mere overskueligt	Vi implementerede ORDER BY i vores SQL-queries
Hvordan kan jeg vise produkter opdelt i kategorier i Flask og HTML templates	At strukturere data i frontend	Vi delte produkter op i Elektronik, Møbler, Personlig pleje og Tøj
Lav en funktion i Flask der sammenligner to kategorier og beregner samlet indtjening	At lave en analysefunktion	Vi brugte SUM (price * quantity) til at sammenligne kategorier
Lav en HTML side med to kolonner og grafer til at sammenligne data visuelt	At forbedre brugeroplevelsen	Vi designede en sammenligningsside med grafer og visuelle elementer
Flyt søgefunktionen fra forsiden til en separat side i Flask appen	At gøre forsiden mere simpel	Vi flyttede søgefeltet til "Alle produkter" siden
Giv mig moderne CSS til en webapp med dark theme, kort-design og flotte knapper	At forbedre design og UI	Vi implementerede et dark theme med cards og gradients
Hvordan laver jeg en centreret forside med tre knapper og velkomsttekst	At forbedre layout og førstehåndsindtryk	Vi brugte flexbox til at centrere indholdet
Hvorfor får jeg fejl i min HTML med Jinja2, og hvordan retter jeg dem	At fejlrette kode	Vi rettede syntax-fejl og template-problemer
Forbedr mit CSS så tabeller og layout ser moderne og professionelt ud	At gøre designet mere professionelt	Vi forbedrede spacing, farver og hover-effekter

AI har været et nyttigt værktøj i vores udviklingsproces, men vi har selv stået for at forstå, tilpasse og implementere løsningerne. Vi har også testet koden løbende, da AI ikke altid giver fejlfrie svar.

Test

I projektet har vi anvendt pytest til at teste programmets funktionalitet. Pytest er et værktøj i Python, som gør det muligt at lave automatiske tests, så man kan undersøge, om programmet virker som det skal. Formålet med testene har været at sikre, at de vigtigste funktioner fungerer korrekt, og at der ikke opstår fejl, når man ændrer i koden.

Vi har brugt både funktionelle tests og integrationstests. De funktionelle tests undersøger, om enkelte dele af programmet virker, mens integrationstestene ser på, hvordan forskellige dele af systemet arbejder sammen, for eksempel forbindelsen mellem hjemmesiden og databasen. Fordelen ved at bruge automatiske tests er, at de kan køres flere gange og hurtigt kan afsløre fejl.

I vores tests har vi blandt andet undersøgt, om siderne kan indlæses korrekt, og om man kan tilføje og slette produkter i databasen. Derudover har vi testet søgefunktionen for at sikre, at den viser de rigtige resultater. Vi har også testet databehandlingen i programmet, herunder beregning af indtjening, antal solgte produkter og gennemsnitspris.

Et eksempel på en test er sammenligning af to kategorier, hvor vi kontrollerer, om systemet beregner de rigtige værdier. Her sammenlignes de beregnede resultater med de forventede værdier for at sikre, at beregningerne er korrekte.

```
PS C:\Users\Nuftalem\OneDrive\Documents\GitHub\Programmering-eksamen> python -m pytest -v
===== test session starts =====
platform win32 -- Python 3.12.10, pytest-9.0.2, pluggy-1.6.0 -- C:\Users\Nuftalem\AppData\Local\Microsoft\WindowsApps\PythonSoftwareFoundation.Python.3.12_qbz5n2kfra8p0\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\Nuftalem\OneDrive\Documents\GitHub\Programmering-eksamen
collected 8 items

tests/test_app.py::test_index_returns_200 PASSED [ 12%]
tests/test_app.py::test_admin_get_returns_200 PASSED [ 25%]
tests/test_app.py::test_admin_post_creates_product_and_redirects PASSED [ 37%]
tests/test_app.py::test_delete_product_removes_product_and_redirects PASSED [ 50%]
tests/test_app.py::test_search_finds_matching_product PASSED [ 62%]
tests/test_app.py::test_allproducts_returns_200 PASSED [ 75%]
tests/test_app.py::test_sammenligning_calculates_correct_result PASSED [ 87%]
tests/test_entity.py::test_product_revenue PASSED [100%]

===== 8 passed in 0.45s =====
```

Figur 8 Pytest

Konklusion

I dette projekt er det lykkedes at udvikle en webbaseret applikation, som kan hjælpe med at analysere og sammenligne indtjening på tværs af produktkategorier. Programmet opfylder de opstillede krav og giver brugeren mulighed for at tilføje, slette, søge og sammenligne produkter på en overskuelig måde.

Jeg har især arbejdet med CRUD-operationer, databehandling og strukturering af kode. Gennem CRUD-operationerne er det muligt at håndtere data i databasen, mens databehandlingen gør det muligt at omdanne rå data til brugbare resultater som indtjening, antal og gennemsnit. Samtidig er koden struktureret efter trelagsmodellen, hvilket gør systemet mere overskueligt og lettere at vedligeholde.

Undervejs i projektet har der været udfordringer i forhold til at få de forskellige dele af systemet til at arbejde sammen, især forbindelsen mellem database og logik. Dette blev løst ved at opdele koden tydeligt og arbejde mere struktureret.

Test af programmet med pytest har vist, at funktionerne virker som forventet, og at systemet håndterer data korrekt. Og hvis systemet skulle videreudvikles, kunne man blandt andet tilføje flere funktioner, såsom redigering af produkter eller mere avancerede analyser. Derudover kunne brugergrænsefladen forbedres yderligere.

Samlet set har projektet givet en bedre forståelse for, hvordan man opbygger en webapplikation med database, og hvordan man arbejder med databehandling og strukturering af kode i praksis.

Bilag

Kode Repository -

<https://github.com/Ollie0618/Programmering-eksamen>